PrepLang

A Data Aware Compiler for Table and Image Preprocessing

Compiler Design Laboratory (BCSE307P)

Phase 3: Final Implementation and Testing

V.Nikhil

24BAI0073

Chapter 1: Introduction

Before data can be used to train a machine learning model it has to be cleaned: missing values are filled or removed, numbers are scaled, rows are filtered, and images are resized and converted. This work is usually done with a Python script that uses pandas for tables and OpenCV for images. Such a script only finds out that something is wrong while it is running. A misspelled column name, a word inside a column of numbers, a broken image file, or decimal pixels saved as a PNG file show up halfway through the data, or never, and then the dataset is silently wrong.

PrepLang is a small language I designed for this job, and this project is its compiler, written in C++17. The compiler does something an ordinary compiler cannot do: it opens the real data files while it is compiling. It reads every cell of a CSV file, only the summary at the end of a Parquet file, and only the header of every image. From this it knows the type of every column, whether values are missing, the smallest and largest value, and the size and channels of every image. It follows these facts through every statement, so it can report data mistakes before anything runs, prove the data checks the programmer wrote, remove work that is not needed, and then write one Python program that does the cleaning.

In Phase 3 I completed the project, tested it with valid, invalid, boundary and large inputs, measured its performance, and added two things: a much faster analysis of large CSV files, and data certificates, which let a compiled program safely accept new data without being compiled again.

Chapter 2: Problem Statement

Data preparation scripts check their assumptions too late. The assumptions are about the data (this column holds numbers, this column has no missing values, every image has three channels, every image has a label), but a script can only test them by running over the data. The existing tools fall into two groups. Data validation tools such as Great Expectations or TensorFlow Data Validation check a dataset when the pipeline runs, separately from the code that transforms it. Static analysis tools look at the code but not at the data, so they cannot know what range a column has or how big an image is.

The problem I solve is: build a compiler for data preprocessing programs that uses the real input data as a compile time input, so that data mistakes are found before processing starts, data contracts are proven or rejected at compile time, and the generated program is both safe and fast. A second problem appears as soon as this works: a compiled program is proven only for the files it was compiled with, while real pipelines run again every day on new files. So the compiler should also say which facts about the data its proofs really need, and the generated program should accept any new file that has those facts.

Chapter 3: Objectives

- Design a small, readable language for table and image preprocessing, with its own tokens and an LL(1) grammar.
- Build every phase of a compiler by hand: lexical analysis, syntax analysis with error recovery, an abstract syntax tree, a symbol table, semantic analysis, three address code, optimization and code generation.
- Use the input data during semantic analysis to type-check every operation and to decide every EXPECT contract as proven, violated or checked at run time.
- Optimize the program using facts about the data, and keep every optimization exact unless the user asks for an approximate one.
- Generate a Python program (pandas and OpenCV) that cannot run on data its proofs do not cover.
- Find the data certificate of every program, so that the compiled program can be reused on new batches of data.
- Keep compile time low even for large inputs.
- Test every module and measure correctness, speed and memory.

Chapter 4: Literature Survey

- **Data validation.** TensorFlow Data Validation (Breck et al., 2019), Deequ (Schelter et al., 2018), Great Expectations and Pandera (Bantilan, 2020) check expectations on a dataset while a pipeline runs. They do not look at the program, so they cannot say at which statement a fact stops being true, and they always check, even when the answer is already known.
- **Static analysis of data science code.** Zhuang and Lu (2022) check column types of data frames by abstract interpretation, and Gerstl, Sihler and Tichy (2026) infer the shape of R data frames, becoming more precise when the datasets are available. They infer names, types or shapes, but not missing values or value ranges, and they do not prove contracts or handle images.
- **Types from data.** F# type providers (Petricek, Guerra and Syme, 2016) read a sample of a data file at compile time to create types. PrepLang reads the whole file (or its metadata), and keeps facts about values, not only types.
- **Compile-time data contracts.** Mirji (2026) checks the structure of Spark pipelines at compile time from declared types. PrepLang proves contracts about values (ranges, missing values, unique keys, foreign keys, image sizes) from the data itself.
- **Hybrid checking.** Soft typing (Cartwright and Fagan, 1991) and hybrid type checking (Flanagan, 2006) prove what they can at compile time and insert run-time checks for the rest. PrepLang applies the same idea to data contracts.
- **Reusing proofs on new inputs.** Proof-carrying code (Necula, 1997) ships a proof with a program. JIT compilers protect fast code with guards and fall back when a guard fails (Hölzle et al., 1992), and databases attach validity ranges to query plans (Markl et al.,

2004). Cousot et al. (2013) infer conditions on the inputs of a program, and DeepInfer (Ahmed et al., 2024) infers data preconditions for neural networks. None of these finds the smallest set of data facts that a compiled data pipeline's correctness proofs depend on, which is what PrepLang's data certificate does.

- **Compiler background.** The phases follow Aho, Lam, Sethi and Ullman, Compilers: Principles, Techniques, and Tools. The analysis follows abstract interpretation (Cousot and Cousot, 1977) with intervals (Cousot and Cousot, 1976).

Chapter 5: System Design

5.1 Architecture

The compiler is a chain of phases. Each phase hands its result to the next one, and if a phase finds an error the compiler stops there, so the same mistake is not reported again by later phases. Figure 1 shows the complete pipeline.

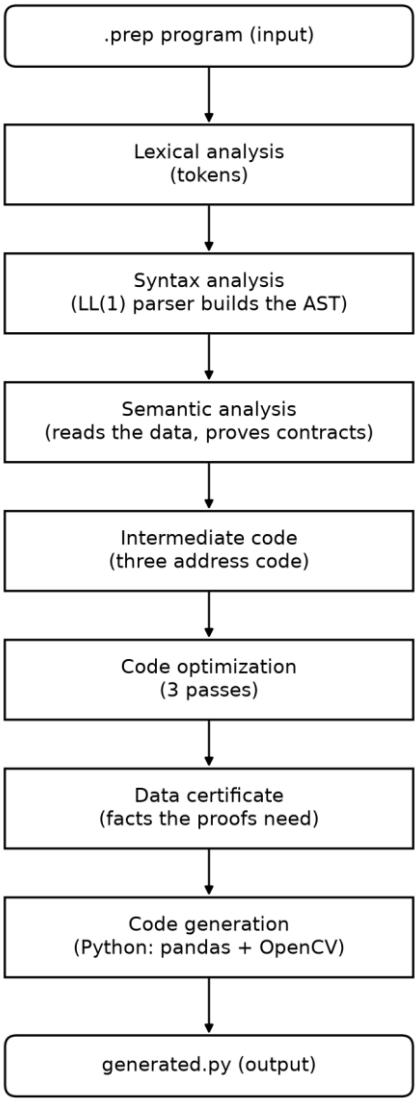


Figure 1: Phases of the PrepLang compiler

5.2 Modules

- **src/common:** source text and diagnostics (line, column, caret, hint, notes).
- **src/lexer:** tokens and the lexer.
- **src/parser:** the recursive descent parser with error recovery.
- **src/ast:** syntax tree nodes, a tree printer and Graphviz export.
- **src/schema:** reading the data: the parallel CSV scanner, Parquet footers, PNG and JPEG headers, CRC-32.
- **src/sema:** the symbol table with the abstract state of the data, selectors, contracts and the semantic analyzer.
- **src/dataset:** lineage of every column and linking images with labels.
- **src/ir:** three address code.

- **src/optimizer:** the pass manager and the optimization passes.
- **src/certify:** data certificates.
- **src/backend:** pandas code, OpenCV code and the planner that joins them.
- **src/explain:** the --explain, --dry-run and --lineage outputs and the run manifest.

5.3 Data Flow

The input is a .prep program. LOAD statements name data files, which the schema module reads during semantic analysis. The analysis keeps an abstract state of every dataset and updates it after every statement. The IR builder turns the checked tree into three address code, the optimizer removes what the state proves unnecessary, the certifier finds the facts the proofs need, and the code generator writes output/generated.py. When generated.py runs it reads the data, checks its fingerprint or certificate, does the cleaning, and writes the output files and a manifest.

Chapter 6: Methodology

I built the compiler phase by phase, testing each phase before starting the next one, as planned in Phase 1. The method rests on four ideas.

1. **Data as a compile time input.** LOAD reads the data while compiling and builds an abstract state: for each column a type, whether values can be missing, an interval of values and an exactness flag; for images the size and channels of every file, the pixel type and the output format.
2. **Transfer functions.** Every statement changes the state in a known way. For example, FILTER marks >= 50 narrows marks to start at 50, FILL_NULL makes a column non-missing, GRAYSCALE makes every image one channel, and NORMALIZE ... RANGE 0 1 makes pixels decimal numbers between 0 and 1.
3. **Three verdicts for contracts.** Each EXPECT is PROVEN when the state shows it always holds, VIOLATED when the data certainly breaks it (the error names the row or file), and otherwise checked when the program runs.
4. **Data certificates.** After optimizing, the compiler analyses the program again many times with weaker facts, and keeps only the facts without which some proof would fail. Figure 2 shows the algorithm.

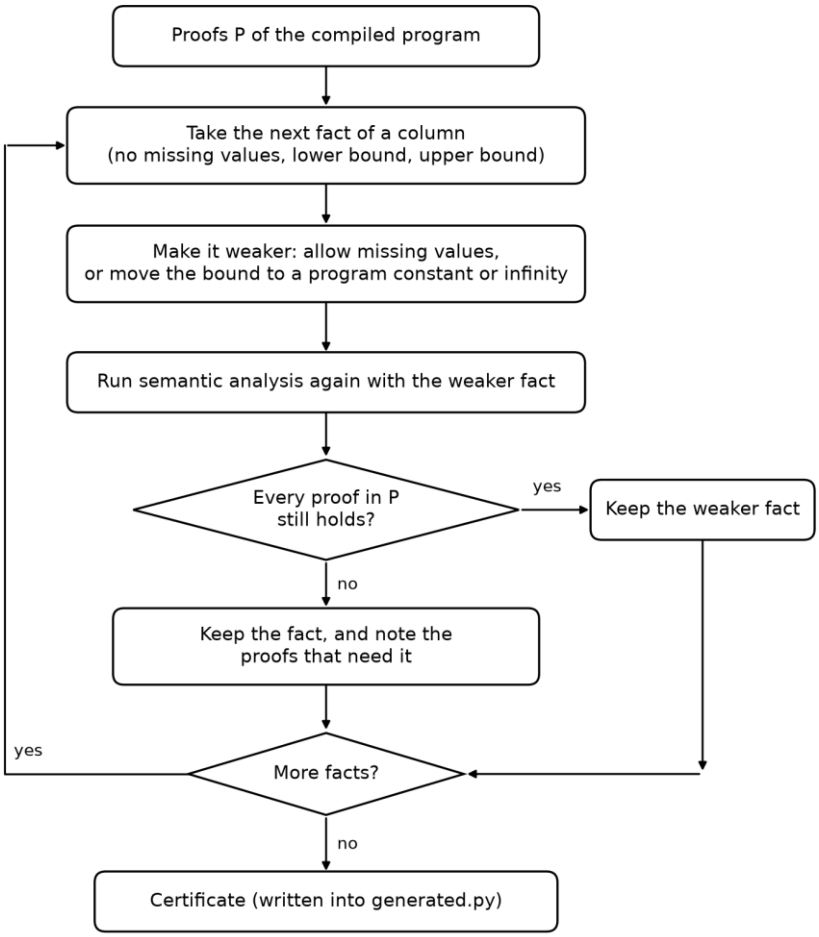


Figure 2: How the data certificate is found

Chapter 7: Compiler Design Concepts Used

7.1 Lexical Analysis

The lexer reads the program one character at a time and groups characters into tokens: keywords (LOAD, FILTER and so on, in any case), identifiers, numbers (with decimals and exponents), strings, operators and punctuation. A # starts a comment. Every token keeps its line and column. An unknown character or a string that is never closed is a lexical error.

7.2 Syntax Analysis

The parser is hand written recursive descent. Every statement starts with its own keyword, so one token of look-ahead decides the rule, which makes the grammar LL(1). Conditions in FILTER follow the usual precedence: OR is weakest, then AND, then NOT, and comparisons bind tightest. On an error the parser uses panic mode recovery: it skips to the next semicolon and continues, so one run reports several errors, and it suggests the closest keyword for a misspelling.

7.3 Abstract Syntax Tree

The parser builds a tree with one node type per statement and per condition. Later phases walk it with the visitor pattern. It can be printed with --ast or saved as a Graphviz picture with --dot.

7.4 Symbol Table

There is one global scope. Each dataset has a name, a kind (TABLE, TABLESET, IMAGE, IMAGESET or DATASET), the line where it was defined, and the current state of its data. Lookups resolve wildcards (company.*.age) and type selectors (WHERE TYPE NUMERIC). A column used after DROP or SELECT removed it is reported, like a use after free.

7.5 Semantic Analysis

Semantic analysis is name checking, type checking and state tracking together. It is a forward abstract interpretation: programs have no loops, so one pass over the statements is enough and no widening is needed. Contracts are decided here, and image save statements are checked against the pixel type and channels.

7.6 Intermediate Code

The checked tree becomes three address code stored as quadruples. Wildcards are expanded into one instruction per table, and conditions are broken into temporaries: t1 = marks >= 50, t2 = attendance > 0.5, t3 = t1 AND t2, FILTER students IF t3.

7.7 Code Optimization

- **No-effect elimination:** removes operations the state proves useless, such as FILL_NULL on a column without missing values or GRAYSCALE on images that already have one channel.
- **Contract check elimination:** removes run-time checks of proven contracts. A contract proven by reading the values keeps a check that runs only when an input changes.
- **Dead code elimination:** removes unused temporaries and image work after the last save.
- **Dead store elimination and column pruning:** a backward liveness analysis over columns, so LOAD reads only the columns that are used.
- **Image streaming:** all image steps become one loop that handles one image at a time.
- **RESIZE fusion:** the only optimization that changes pixels, so it runs only with --approx.

7.8 Target Code Generation

The execution planner sends table instructions to the pandas code generator and image instructions to the OpenCV code generator, in program order, and writes one Python file. Column types become pandas nullable types, so missing values behave exactly as the analysis assumed.

7.9 Error Detection and Recovery

Every message names the phase, the line and column, draws a caret under the spot and gives a hint. The parser recovers from syntax errors, and one mistake in a wildcard is reported once with a note for every table it affects. The generated program reports run-time errors with the line of the statement and, for refused inputs, the row and the proof.

Chapter 8: Implementation

8.1 Language, Tools and Libraries

- C++17 with Visual Studio 2022 Build Tools (MSVC) and CMake; no external C++ libraries.
- Python 3 with pandas, NumPy, OpenCV and pyarrow for the generated programs and the evaluation.
- Visual Studio Code tasks for building, running, explaining and testing; Git and GitHub for version history.
- About 8,500 lines of C++ in src, 181 unit tests in 9 test programs, 8 example programs and 5 error examples.

8.2 The Parallel CSV Scanner

In Phase 2 the compiler read a CSV file line by line, copying every cell into a new string, and one million rows took 2.04 seconds, about three times longer than pandas needs to read the file. I rewrote this part as a parallel scanner, shown in Figure 3. The file is read once into memory. Its rows are cut into one piece per processor core, always at a line break, and each piece is analysed by its own thread. Cells are looked at in place with std::string_view instead of being copied, and the type of a cell and the value of a number are found in one pass. For numbers I used Clinger's fast path: when the digits fit exactly in a double and the power of ten is at most 10^22, one multiplication or division gives the correctly rounded result, and every other number is handed to std::from_chars. The CRC-32 fingerprint is computed on its own thread at the same time, eight bytes per step.

The pieces are merged in file order. The first problem in the file wins, and the line where a column first turns into text is found from the first text cell and the first boolean and number cells of each piece, so the messages name exactly the same lines as a single pass would.

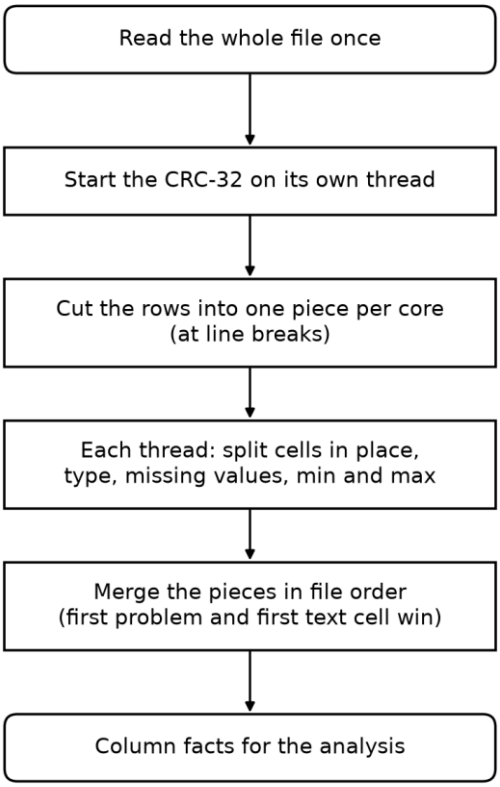


Figure 3: The parallel CSV scanner

Now one million rows take 0.13 seconds to analyse, including starting the compiler. That is 4.7 times faster than pandas reads the same file with its default reader (0.62 s), and about as fast as pandas with the multi-threaded pyarrow reader (0.11 s, measured inside an already running Python), although the compiler also checks the encoding, infers every type and finds every minimum, maximum and missing value.

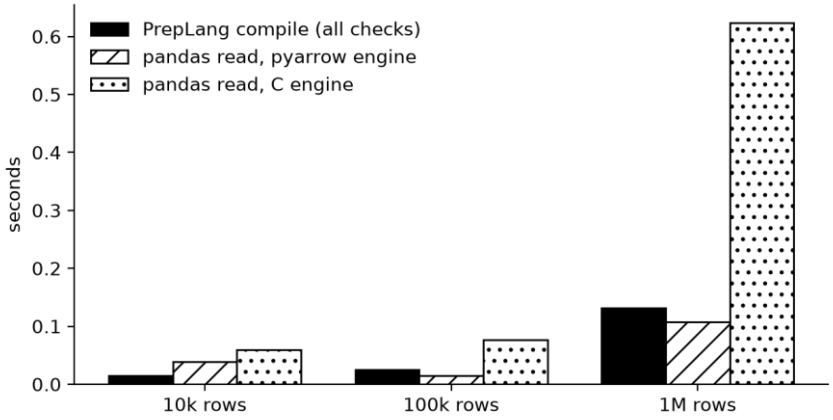


Figure 4: Compile time of PrepLang against just reading the CSV file in Python

8.3 Data Certificates

The certifier takes the facts of every input table and first clears the exactness flag, so they describe a set of possible files instead of one file. It checks that these facts still prove everything the compiled program uses. Then, for every column the optimized program reads, it tries to make one fact weaker and runs the whole semantic analysis again: it allows missing values, and it moves the lower and upper bounds to minus or plus infinity, or else to the loosest constant of the program that still works. A weaker fact is kept when no proof and no removed operation is lost. Otherwise the fact stays in the certificate together with the proofs that needed it. The analysis compares ranges only with the program's own constants, so no other bound could do better.

These re-analyses read no data: the tables are replaced by their facts and image headers are kept in memory. For the students example the whole certificate needs 13 extra analyses and takes a few milliseconds. The result is printed with --certificate and written into the generated program:

```
students  (compiled with examples/tabular/students.csv)
  name        STRING   no missing values   <- line 6: EXPECT students.name NOT_NULL
  age         INTEGER  (any values)
  marks       INTEGER  >= 0               <- line 5: EXPECT students.marks BETWEEN 0 AND 100
                       <= 100             <- line 5: EXPECT students.marks BETWEEN 0 AND 100
  attendance  FLOAT    (any values)
```

The file had marks between 35 and 95, but the only proof about marks needs 0 to 100, so a new file with a mark of 100 is accepted. The run can be given a new file on the command line, for example py -3 output/generated.py students=new_students.csv. Figure 4 shows what the generated program does with each input. Contracts proven by reading the values themselves (UNIQUE, REFERENCES and MATCH_RATE) keep their check, but it runs only when an input is not the compiled file.

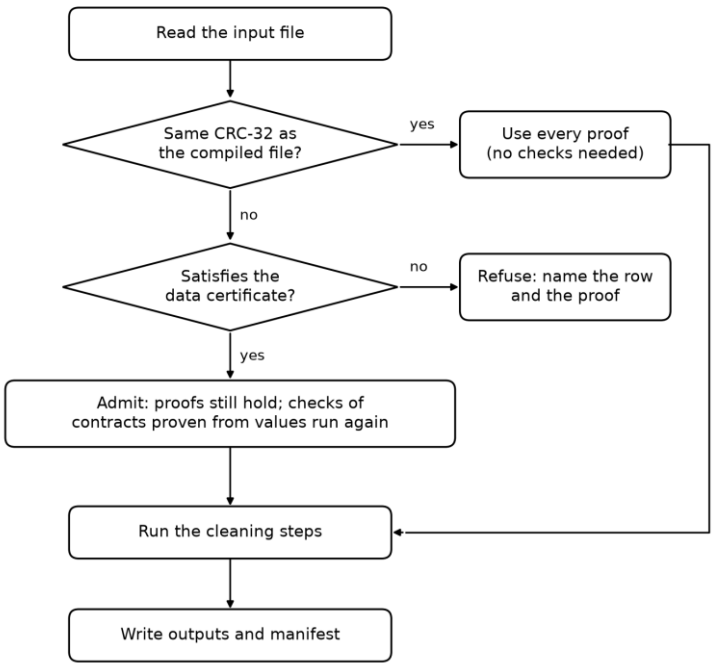


Figure 5: What the generated program does with an input file

Why this is safe: every transfer function is monotone once exactness is cleared, and every proof or removal is a test of the form "the state lies inside a fixed condition". A file that satisfies the certificate has a state inside the certified state, so the analysis of that file would prove at least everything the compiled program relies on. I wrote this argument as a theorem in the paper and also check it with a test: 400 random files are tried, and every file the certificate accepts, when compiled on its own, keeps every proof.

8.4 A Bug Found by the Certificate Experiment

While evaluating certificates I found a bug from an earlier phase. The run-time MATCH_RATE check compared the labels with every image file, including a broken image that REMOVE_CORRUPT had removed before the LINK, while the compile-time proof had left it out. Before Phase 3 this check was always removed as proven, so it never ran. Now it runs when the labels change, and it failed where it should pass. The LINK now records the files removed before it and the generated check skips the same files. I added a regression test for it.

Chapter 9: Testing and Results

9.1 Unit Tests

All 181 unit tests in 9 test programs pass with ctest (Figure 5):

- **test_lexer** 15 tests: tokens, keywords, numbers, strings and lexical errors.
- **test_parser** 29 tests: every statement, operator precedence, error recovery and suggestions.
- **test_semantic** 29 tests: types, names, missing values, ranges and contracts.
- **test_tableset** 13 tests: folders of tables, wildcards and contracts across tables.
- **test_imageset** 21 tests: image headers, sizes, channels, image errors and MATCH_RATE.
- **test_parquet** 23 tests: Parquet footers, statistics and contracts.
- **test_optimizer** 12 tests: every optimization.
- **test_scanner** 22 tests: the parallel scanner gives the same facts with 1 to 16 pieces, reports the first problem of the file, and parses 20,000 random numbers exactly like std::from_chars.
- **test_certificate** 17 tests: which facts are certified and why, admission and refusal, conditional checks, and the soundness property on 400 random files.

```
> ctest --test-dir build -C Release
    Start 1: lexer
1/9 Test #1: lexer ............................   Passed    0.01 sec
    Start 2: parser
2/9 Test #2: parser ...........................   Passed    0.02 sec
    Start 3: semantic
3/9 Test #3: semantic .........................   Passed    0.04 sec
    Start 4: tableset
4/9 Test #4: tableset .........................   Passed    0.03 sec
    Start 5: imageset
5/9 Test #5: imageset .........................   Passed    0.04 sec
    Start 6: parquet
6/9 Test #6: parquet ..........................   Passed    0.02 sec
    Start 7: optimizer
7/9 Test #7: optimizer ........................   Passed    0.02 sec
    Start 8: scanner
8/9 Test #8: scanner ..........................   Passed    0.59 sec
    Start 9: certificate
9/9 Test #9: certificate ......................   Passed    1.30 sec

100% tests passed out of 9

Total Test time (real) =   2.07 sec
```

Figure 6: All test programs pass

9.2 Test Cases

Each case below was run by docs/review3/cases.py, which records the real output, so the actual results are copied from the program and not typed by hand. All of them behave as expected.

9.3 Valid inputs

Test 1. Compile and run examples/tabular/students.prep.

Expected: the cleaned file is written. **Actual:** Saved 4 rows to output\clean.csv

Test 2. Compile examples/tabular/tableset.prep (a folder of four tables).

Expected: compiles without errors. **Actual:** Compiled successfully: 0 error(s), 0 warning(s).

Test 3. Compile examples/tabular/metadata_only.prep (Parquet, footer only).

Expected: contracts proven from the footer statistics. **Actual:** contracts proven 2

Test 4. Compile examples/images/linked.prep (images linked to labels).

Expected: compiles without errors. **Actual:** Compiled successfully: 0 error(s), 0 warning(s).

9.4 Invalid inputs

Test 5. A character the language does not have ($).

Expected: lexical error. **Actual:** error[lexical]: unexpected character '$'

Test 6. A misspelled keyword (NORMALISE).

Expected: syntax error with a suggestion. **Actual:** = hint: did you mean 'NORMALIZE'?

Test 7. NORMALIZE on a text column.

Expected: type error. **Actual:** error[type]: NORMALIZE needs a numeric column, but s.name is STRING

Test 8. A column used after DROP removed it.

Expected: semantic error naming the DROP. **Actual:** error[semantic]: column 'marks' was removed from 's' at line 2

Test 9. A contract the data breaks (marks BETWEEN 0 AND 90).

Expected: compile error naming the row. **Actual:** error[contract]: contract violated: s.marks has values from 0 to 100; row 2 of s.csv has 100

9.5 Boundary cases

Test 10. An empty CSV file.

Expected: schema error. **Actual:** error[schema]: CSV file 'e.csv' is empty

Test 11. A CSV row with one value too many.

Expected: schema error with the line number. **Actual:** error[schema]: line 3 of 'x.csv' has 3 values but the header has 2 columns

Test 12. A CSV file saved as Windows-1252, not UTF-8.

Expected: schema error with line and byte. **Actual:** error[schema]: line 2 of 'w.csv' is not UTF-8 text (byte 0xE9)

Test 13. BETWEEN 0 AND 100 when the data has exactly 0 and 100.

Expected: PROVEN (both ends included). **Actual:** line 2: EXPECT s.marks BETWEEN 0 AND 100: PROVEN, values are in [0, 100]

Test 14. FILTER marks > 100 when the largest mark is 100.

Expected: warning: removes every row. **Actual:** warning[semantic]: FILTER removes every row: the condition is never true

9.6 Error conditions when the generated program runs

Test 15. The compiled program given a new batch inside the certificate.

Expected: admitted and run. **Actual:** Admitted next.csv for s: it satisfies the data certificate

Test 16. The compiled program given a batch with marks 130.

Expected: refused, naming the row and the proof. **Actual:** ContractError: bad.csv is not the file the program was compiled with, and marks is 130 in data row 2, above 100; the compiled program relies on this for line 2: EXPECT s.marks BETWEEN 0 AND 100. Compile the .prep file again for this input.

Test 17. A deferred contract that fails when the program runs.

Expected: ContractError naming the line. **Actual:** ContractError: line 3: s.marks has values outside 50 to 100

9.7 Output Evidence

```
> preplang examples/tabular/students.prep --explain
PREPLANG EXECUTION PLAN

INPUTS
------
  students        TABLE     examples/tabular/students.csv, 8 rows, 6 columns

STATIC ANALYSIS
---------------
  schemas inferred              1
  contracts proven              3
  contracts violated            0
  contracts checked at run time 1
  semantic errors               0
  warnings                      0

OPTIMIZATIONS
-------------
  contract check elimination    2
  instructions                  16 -> 14

DATA CERTIFICATE
----------------
  columns read                  6 (type fixed)
  value facts the proofs need   3 (see --certificate)
  re-checked if inputs change   0
  other inputs                  admitted by generated.py table=file.csv

BACKENDS
--------
  tables                        Python + pandas (14 instructions)

No files have been modified.

Compiled successfully: 0 error(s), 0 warning(s).
```

Figure 7: Execution plan of examples/tabular/students.prep (--explain)

```
> preplang examples/errors/semantic_error.prep
error[type]: NORMALIZE needs a numeric column, but students.name is STRING
  --> examples/errors/semantic_error.prep:5:11
  |
5 | NORMALIZE students.name;                      # type: name is STRING
  |           ^

error[type]: cannot compare INTEGER column 'marks' with text "excellent"
  --> examples/errors/semantic_error.prep:6:23
  |
6 | FILTER students WHERE marks > "excellent";    # type: INTEGER compared with text
  |                       ^
  = hint: use a whole number, e.g. 0

error[type]: FILL_NULL with text "unknown" needs a matching column type, but students.age is INTEGER
  --> examples/errors/semantic_error.prep:7:11
  |
7 | FILL_NULL students.age WITH "unknown";        # type: INTEGER column filled with text
  |           ^
  = hint: use a whole number, e.g. 0

error[semantic]: column 'mark' does not exist in 'students'
  --> examples/errors/semantic_error.prep:8:11
  |
8 | NORMALIZE students.mark;                      # semantic: no column 'mark'
  |           ^
  = hint: did you mean 'marks'?

error[contract]: contract violated: students.department has 1 missing value in
  examples/tabular/students.csv
  --> examples/errors/semantic_error.prep:9:8
  |
9 | EXPECT students.department NOT_NULL;          # contract: department has an empty cell
  |        ^
  = hint: use FILL_NULL or DROP_NULL before this EXPECT

error[contract]: contract violated: students.marks has values from 35 to 95; row 1 of
  examples/tabular/students.csv has 85
   --> examples/errors/semantic_error.prep:10:8
   |
10 | EXPECT students.marks BETWEEN 0 AND 50;       # contract: marks go up to 95
   |        ^
   = hint: every value must be between 0 and 50

error[semantic]: column 'age' was removed from 'students' at line 12
   --> examples/errors/semantic_error.prep:13:23
   |
13 | FILTER students WHERE age > 20;               # semantic: age was dropped above
   |                       ^
   = hint: use the column before that line, or keep it in the SELECT/DROP

error[semantic]: dataset 'student' is not defined
   --> examples/errors/semantic_error.prep:14:6
   |
14 | SAVE student AS "out.csv";                    # semantic: no dataset 'student'
   |      ^
   = hint: did you mean 'students'?
```

Figure 8: Semantic errors with line, caret and hint

```
> preplang float_png.prep   (linked.prep with CONVERT imgs TO PNG)
error[type]: cannot save FLOAT32 pixels as PNG: NORMALIZE at line 10 scaled them to [0, 1]
   --> examples/images/float_png.prep:13:1
   |
13 | SAVE_IMAGES imgs TO "processed/ml_ready/";
   | ^
   = hint: image files store 8-bit pixels, so these would come out almost black; add CONVERT imgs TO
   NPY; to keep the float values
```

Figure 9: Decimal pixels saved as PNG are stopped at compile time

```
> preplang examples/tabular/students.prep --certificate
==================== DATA CERTIFICATE (Certifier) ====================
The compiled program stays correct for any input with these facts
(found by analysing the program 13 more times with weaker facts):

students  (compiled with examples/tabular/students.csv)
  name        STRING   no missing values   <- line 6: EXPECT students.name NOT_NULL
  age         INTEGER  (any values)
  marks       INTEGER  >= 0                <- line 5: EXPECT students.marks BETWEEN 0 AND 100
                       <= 100              <- line 5: EXPECT students.marks BETWEEN 0 AND 100
  attendance  FLOAT    (any values)
  passed      BOOLEAN  (any values)
  department  STRING   (any values)
```

Figure 10: The data certificate of the students program

```
> py -3 output/generated.py students=next_week.csv
Admitted eval\work\evidence\next_week.csv for students: it satisfies the data certificate
Saved 4 rows to output\clean.csv
Wrote output\preplang_manifest.json

> py -3 output/generated.py students=typo.csv
ContractError: eval\work\evidence\typo.csv is not the file the program was compiled with, and marks
  is 104 in data row 4, above 100; the compiled program relies on this for line 5: EXPECT
  students.marks BETWEEN 0 AND 100. Compile the .prep file again for this input.
```

Figure 11: The compiled program accepts a new batch and refuses a wrong one

9.8 Performance and Evaluation

- **Injected mistakes:** 24 of 24 mistakes injected into the example programs are reported before any data is processed, with no false alarms; a script that does the same steps crashes part-way on most of them and silently writes wrong output on the rest.
- **Contracts:** 15 of the 16 contracts in the examples are decided at compile time.
- **Optimizations:** on a table of 200,000 rows and 40 columns, the optimized program reads 5 columns and runs no checks, and takes 1.49 s against 1.81 s without optimization, with the same output. For 1,000 images, streaming needs 10 MiB of memory instead of 922 MiB. RESIZE fusion saves little and changes pixels, so it is left to --approx.
- **Compile time:** one million CSV rows in 0.13 s (Section 8.2); a Parquet file with 16 million rows in 0.02 s, because only its footer is read.
- **Large inputs:** on 3,000 images, saving decimal pixels as PNG is reported in 0.26 s, while the equivalent script runs for 8.54 s and writes 3,000 black images.
- **Real failures:** of 44 data preparation failures collected from GitHub issues, PrepLang reports 18 (41%) before any data is processed.
- **Data certificates:** seven compiled programs were given 244 new batches of data (other days of a sensor process, resampled rows, values that drift, missing values and values out of range), and every batch was also compiled from scratch as the ground truth. The certificate admitted 209 batches (86%); taking all facts of the compiled file, as a schema inferred by TFDV would, admits only 96 (39%), and the plain fingerprint admits none. 209 of the 209 admitted runs ended exactly like the fresh compilation (the same output files, or stopped by the same contract), and every refused batch also failed to compile from scratch, so nothing was refused that could have run.

Chapter 10: Conclusion

PrepLang shows that, for batch data preparation, treating the input data as a compile time input is worth it. The compiler has every classic phase, written by hand, and its semantic analysis carries real facts about the data through the program. This turns most data mistakes into compile time errors with clear messages, decides almost every contract before the program runs, and justifies optimizations that keep the output exact. In Phase 3 the analysis of large CSV files became faster than reading them with pandas, and data certificates removed the main limit of

the earlier design: a compiled program is no longer tied to one file, but accepts any new batch that has the few facts its proofs need, and explains exactly why when it refuses one.

Chapter 11: Future Enhancements

- Certificates for image folders, by checking the headers of new images against the certified sizes and channels.
- Joins, date columns and per-image label files, which several of the real GitHub failures needed.
- A native C++ backend for the table operations, so the generated program does not need Python.
- Searching for the smallest certificate instead of a greedy one, and certificates for groups of columns.
- A language server for VS Code that shows the state of the data while typing.

References

[1] A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed., Pearson, 2006.

[2] P. Cousot and R. Cousot, "Abstract interpretation: a unified lattice model for static analysis of programs," POPL 1977.

[3] P. Cousot and R. Cousot, "Static determination of dynamic properties of programs," Int. Symp. on Programming, 1976.

[4] E. Breck, N. Polyzotis, S. Roy, S. E. Whang and M. Zinkevich, "Data validation for machine learning," MLSys 2019.

[5] S. Schelter et al., "Automating large-scale data quality verification," PVLDB 11(12), 2018.

[6] N. Bantilan, "pandera: Statistical data validation of pandas dataframes," SciPy 2020.

[7] T. Petricek, G. Guerra and D. Syme, "Types from data: Making structured data first-class citizens in F#," PLDI 2016.

[8] Y. Y. Zhuang and M.-Y. Lu, "Enabling type checking on columns in data frame libraries by abstract interpretation," IEEE Access, 2022.

[9] O. Gerstl, F. Sihler and M. Tichy, "Inferring the shape of data frames in R programs using abstract interpretation," arXiv:2607.03889, 2026.

[10] V. Mirji, "Shift schema drift left: policy-aware compile-time contracts for typed JVM and Spark pipelines," arXiv:2604.16986, 2026.

[11] C. Flanagan, "Hybrid type checking," POPL 2006.

[12] R. Cartwright and M. Fagan, "Soft typing," PLDI 1991.

[13] G. C. Necula, "Proof-carrying code," POPL 1997.

[14] U. Hölzle, C. Chambers and D. Ungar, "Debugging optimized code with dynamic deoptimization," PLDI 1992.

[15] V. Markl et al., "Robust query processing through progressive optimization," SIGMOD 2004.

[16] P. Cousot, R. Cousot, M. Fähndrich and F. Logozzo, "Automatic inference of necessary preconditions," VMCAI 2013.

[17] S. Ahmed, H. Gao and H. Rajan, "Inferring data preconditions from deep learning models for trustworthy prediction in deployment," ICSE 2024.

[18] W. D. Clinger, "How to read floating point numbers accurately," PLDI 1990.

[19] Apache Parquet file format documentation, https://parquet.apache.org/docs/

[20] pandas documentation, https://pandas.pydata.org/docs/ ; OpenCV documentation, https://docs.opencv.org/

Appendix A: Grammar

The grammar in EBNF, copied from the project README when this report is built. Every statement starts with a different keyword, so the grammar is LL(1).

```
program        = { statement } EOF ;
statement      = load | drop_null | fill_null | normalize | filter | select | drop | save | expect
               | resize | grayscale | compress | convert | remove_corrupt | link ;


load           = ( "LOAD" | "LOAD_TABLES" | "LOAD_IMAGE" | "LOAD_IMAGES" ) STRING "AS" IDENT [ "RECURSIVE" ] ";" ;
save           = "SAVE" table_ref "AS" STRING ";"  | "SAVE_TABLES" IDENT "TO" STRING ";"
               | "SAVE_IMAGE" IDENT "AS" STRING ";" | "SAVE_IMAGES" IDENT "TO" STRING ";" ;
drop_null      = "DROP_NULL" table_selector ";" ;
fill_null      = "FILL_NULL" column_selector "WITH" literal [ "IF_EXISTS" ] ";" ;
normalize      = "NORMALIZE" column_selector ( "RANGE" number number
                                             | [ "USING" ( "ZSCORE" | "MINMAX" ) ] ) [ "IF_EXISTS" ] ";" ;
filter         = "FILTER" table_ref "WHERE" or_expr ";" ;
select         = "SELECT" table_ref column_list ";" ;
drop           = "DROP" table_selector column_list [ "IF_EXISTS" ] ";" ;
expect         = "EXPECT" column_selector constraint [ "IF_EXISTS" ] ";" ;
constraint     = "NUMBER" | "STRING" | "BOOLEAN" | "NOT_NULL" | "BETWEEN" number "AND" number | "UNIQUE"
               | "REFERENCES" path | "SCHEMA_SAME_AS" table_ref [ "EXCEPT" column_list ]
               | "CHANNELS" integer | "MIN_WIDTH" integer | "MIN_HEIGHT" integer | "MATCH_RATE" number "%" ;
resize         = "RESIZE" IDENT "TO" integer integer [ "KEEP_ASPECT" ] ";" ;
grayscale      = "GRAYSCALE" IDENT ";" ;
compress       = "COMPRESS" IDENT "QUALITY" integer ";" ;
convert        = "CONVERT" IDENT "TO" ( "JPG" | "PNG" | "NPY" ) ";" ;
remove_corrupt = "REMOVE_CORRUPT" IDENT ";" ;
link           = "LINK" IDENT "WITH" IDENT "ON" IDENT "AS" IDENT ";" ;


or_expr        = and_expr { "OR" and_expr } ;
and_expr       = not_expr { "AND" not_expr } ;
not_expr       = "NOT" not_expr | primary ;
primary        = "(" or_expr ")" | IDENT comparison_op literal ;


table_ref       = IDENT [ "." IDENT ] ;
table_selector  = IDENT [ "." ( IDENT | "*" ) ] ;
column_selector = IDENT { "." ( IDENT | "*" ) }
                  [ "WHERE" ( "TYPE" ( "NUMERIC" | "NUMBER" | "STRING" | "BOOLEAN" ) | "NULLABLE" ) ]
                  [ "EXCEPT" column_list ] ;
path            = IDENT { "." IDENT } ;
column_list     = "[" IDENT { "," IDENT } "]" ;
literal         = number | STRING | "TRUE" | "FALSE" ;
number          = [ "-" ] NUMBER ;
```

Appendix B: User Instructions

One-time setup: install Visual Studio 2022 Build Tools (Desktop development with C++), CMake 3.16 or newer, and Python 3 with pandas, NumPy, OpenCV and pyarrow (py -3 -m pip install pandas numpy opencv-python pyarrow). Open the PrepLang folder in VS Code and accept the recommended extensions.

1. Build the compiler: press Ctrl + Shift + B, or run the commands below.
2. Open a .prep file and choose Terminal, Run Task, "PrepLang: compile and run the open .prep file". Every phase is printed and the cleaned data appears in the output folder.
3. To see the plan without running anything, use the task "PrepLang: explain the open .prep file".
4. To see which facts the program needs, add --certificate. To run the compiled program on a new file, pass table=file, for example students=new_students.csv.
5. To see the error messages, open a file in examples/errors and run the compile task.
6. To run the tests, choose the task "PrepLang: run all tests".

```
cmake -S . -B build -G "Visual Studio 17 2022"
cmake --build build --config Release
build\Release\preplang.exe examples\tabular\students.prep --run
build\Release\preplang.exe examples\tabular\students.prep --certificate
py -3 output\generated.py students=new_students.csv
ctest --test-dir build -C Release
```

The full source code, the evaluation scripts and the research paper draft are in the project repository at github.com/Nikhi00718/preLang.